A Better Encryption Standard (BES-512)

Luke Hawkins, Matthew Michal,
Jeffrey Sims, Moozoo Vang,
Jakob Walker, Trin Wasinger

2025-03-12

Abstract

The report outlines the approach to developing an encryption algorithm that is more resistant to quantum computing based brute force attacks than AES-256. The weaknesses of AES-256 are identified and a new algorithm is presented that accounts for those weaknesses and addresses the challenges associated with expanding the key size. The proposed algorithm is an AES variant with a 512 bit key size along with other expanded components like plaintext bit size, mix columns matrix, shift rows operation, and number of rounds. By doubling the key size to 512 bits, the algorithm becomes significantly more secure than AES-256. The drastic increase in security is critical to ensuring the algorithm is resistant to brute force attacks from quantum hardware using Grover's algorithm. Originally, the group intended to verify the performance of BES-512 through NIST's Cryptographic Algorithm Verification Program (CAVP) and Automated Cryptographic Validation Test Systems (ACVTS). However, the NIST systems were not conducive to experimentation with homebrew algorithms, like BES-512. To fill the need for a testing suite, a novel cryptographic validation suite that leverages machine learning was developed. The suite's accuracy is validated with existing algorithms and utilized to determine the security of the BES-512 algorithm. The algorithm was successfully implemented in Rust, demonstrating improved avalanche effect and Tau correlation over AES-256 and AES-128.

Introduction

The encryption landscape requires change, with error correcting quantum computing on the horizon. Although current implementations of AES-256 appear to be resilient to quantum attacks [1] and AES-256 has been designated as quantum resistant, the group is aware of the rapid development and expanding logical qubit counts of modern quantum hardware, which begets the need for an algorithm that is even more quantum resistant than AES-256. The group speculates that in the future, a sufficiently powerful quantum computer could have the capability of breaking the symmetric encryption algorithms that society relies on. One of the most important symmetric encryption algorithms, AES, could be at risk of becoming insecure. Even the AES-256, which uses a 256 bit key to encrypt and decrypt data, could be compromised through advanced brute force attacks that utilize Grover's algorithm on future hardware. To address this concern, the group is proposing an AES variant with

double the key size of AES-256, a symmetric AES-like algorithm utilizing a 512 bit key. The proposed algorithm expands the key size to make brute force attacks significantly more difficult, including those leveraging Grover's algorithm. In order to enable the expanded key size, some of the AES operations are modified, breaking away from the AES standard which makes the new algorithm considered a variant of AES. As discussed later in the proposal, the mix columns function, shift rows operation, number of rounds, and plaintext bit size are modified to outline the most secure standard to employ a 512 bit key.

Related Work

The original AES design, otherwise known as the Rijidael cipher, was created during the 2000 Advanced Encryption Software competition, where different algorithms for encryption/decryption were created and tested against one another [2]. This was needed because the previous encryption standard, DES, was becoming vulnerable to brute-force attacks with the growth in computational power each year. Each cipher was tested and compared based on a variety of factors, ranging from performance, security and feasibility to implement, with the Rijndael cipher being chosen as the official AES. It can use 128, 192 or 256 bit keys to encrypt plaintext, as compared to the team's proposed 512 bit solution. At the time, AES-128 was deemed sufficiently secure against existing brute-force and cryptographic attacks, but as highlighted previously, the rapid evolution of quantum computing introduces some doubt as to whether AES will still be secure in the coming decades.

The whirlpool hashing function, created by Paulo S.L.M. Barreto and Vincent Rijmen, is a one way hash function designed similarly to AES [3]. This hash function has a block size of 512 bits, and contains a similar round structure as AES, with a Substitute Bytes, Shift Columns, Mix Rows, and an Add Round Key Phase. This cryptographic algorithm, while not the same as the team's proposed 512 bit implementation of BES, is similar in that it modifies the original AES algorithm to achieve improved security. One major difference is that the whirlpool hashing algorithm is designed to be fast, lightweight, and usable on small embedded systems, while BES is designed to take up more storage and memory but increase the overall security. Another major difference is the fact that the whirlpool hash function is a one way function, while BES needs to work in both directions. While the whirlpool hash function shares the same inspiration as BES, it differs from BES in that whirlpool is an AES inspired hash function.

Key Idea and Approach

Mix Columns Matrix

$$\begin{bmatrix} 2 & 3 & 4 & 5 & 6 & 7 & 1 & 1 \\ 1 & 2 & 3 & 4 & 5 & 6 & 7 & 1 \\ 1 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ 7 & 1 & 1 & 2 & 3 & 4 & 5 & 6 \\ 6 & 7 & 1 & 1 & 2 & 3 & 4 & 5 \\ 5 & 6 & 7 & 1 & 1 & 2 & 3 & 4 \\ 4 & 5 & 6 & 7 & 1 & 1 & 2 & 3 \\ 3 & 4 & 5 & 6 & 7 & 1 & 1 & 2 \end{bmatrix} \quad \begin{bmatrix} 4 & 5 & 6 & 7 & 1 & 1 & 2 & 3 \\ 3 & 4 & 5 & 6 & 7 & 1 & 1 & 2 \\ 2 & 3 & 4 & 5 & 6 & 7 & 1 & 1 \\ 1 & 2 & 3 & 4 & 5 & 6 & 7 & 1 \\ 1 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ 7 & 1 & 1 & 2 & 3 & 4 & 5 & 6 \\ 6 & 7 & 1 & 1 & 2 & 3 & 4 & 5 \\ 5 & 6 & 7 & 1 & 1 & 2 & 3 & 4 \end{bmatrix}$$

Figure 1: Possible M Matrices

$$\begin{bmatrix} 4 & 5 & 6 & 7 & 1 & 1 & 2 & 3 \\ 3 & 4 & 5 & 6 & 7 & 1 & 1 & 2 \\ 2 & 3 & 4 & 5 & 6 & 7 & 1 & 1 \\ 1 & 2 & 3 & 4 & 5 & 6 & 7 & 1 \\ 1 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ 7 & 1 & 1 & 2 & 3 & 4 & 5 & 6 \\ 6 & 7 & 1 & 1 & 2 & 3 & 4 & 5 \\ 5 & 6 & 7 & 1 & 1 & 2 & 3 & 4 \end{bmatrix} \quad \begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 & 13 & 14 & 15 & 16 \\ 17 & 18 & 19 & 20 & 21 & 22 & 23 & 24 \\ 25 & 26 & 27 & 28 & 29 & 30 & 31 & 32 \\ 33 & 34 & 35 & 36 & 37 & 38 & 39 & 40 \\ 41 & 42 & 43 & 44 & 45 & 46 & 47 & 48 \\ 49 & 50 & 51 & 52 & 53 & 54 & 55 & 56 \\ 57 & 58 & 59 & 60 & 61 & 62 & 63 & 64 \end{bmatrix}$$

Figure 2: Possible M Matrix multiplied by a test matrix

- (3,1) = 2 * 1 xor 3 * 9 xor 4 * 17 xor 5 * 25 xor 6 * 33 xor 7 * 41 xor 1 * 49 xor 1 * 57
- (3,2) = 2 * 2 xor 3 * 10 xor 4 * 18 xor 5 * 26 xor 6 * 34 xor 7 * 42 xor 1 * 50 xor 1 * 58
- (3,3) = 2 * 3 xor 3 * 11 xor 4 * 19 xor 5 * 27 xor 6 * 35 xor 7 * 43 xor 1 * 51 xor 1 * 59
- (3,4) = 2 * 4 xor 3 * 12 xor 4 * 20 xor 5 * 28 xor 6 * 36 xor 7 * 44 xor 1 * 52 xor 1 * 60
- (3,5) = 2 * 5 xor 3 * 13 xor 4 * 21 xor 5 * 29 xor 6 * 37 xor 7 * 45 xor 1 * 53 xor 1 * 61
- (3,6) = 2 * 6 xor 3 * 14 xor 4 * 22 xor 5 * 30 xor 6 * 38 xor 7 * 46 xor 1 * 54 xor 1 * 62
- (3,7) = 2 * 7 xor 3 * 15 xor 4 * 23 xor 5 * 31 xor 6 * 39 xor 7 * 47 xor 1 * 55 xor 1 * 63
- (3,8) = 2 * 8 xor 3 * 16 xor 4 * 24 xor 5 * 32 xor 6 * 40 xor 7 * 48 xor 1 * 56 xor 1 * 64
- (4,1) = 1 * 1 xor 2 * 9 xor 3 * 17 xor 4 * 25 xor 5 * 33 xor 6 * 41 xor 7 * 49 xor 1 * 57
- (4,2) = 1 * 2 xor 2 * 10 xor 3 * 18 xor 4 * 26 xor 5 * 34 xor 6 * 42 xor 7 * 50 xor 1 * 58
- (4,3) = 1 * 3 xor 2 * 11 xor 3 * 19 xor 4 * 27 xor 5 * 35 xor 6 * 43 xor 7 * 51 xor 1 * 59
- (4,4) = 1 * 4 xor 2 * 12 xor 3 * 20 xor 4 * 28 xor 5 * 36 xor 6 * 44 xor 7 * 52 xor 1 * 60
- (4,5) = 1 * 5 xor 2 * 13 xor 3 * 21 xor 4 * 29 xor 5 * 37 xor 6 * 45 xor 7 * 53 xor 1 * 61
- (4,6) = 1 * 6 xor 2 * 14 xor 3 * 22 xor 4 * 30 xor 5 * 38 xor 6 * 46 xor 7 * 54 xor 1 * 62
- (4,7) = 1 * 7 xor 2 * 15 xor 3 * 23 xor 4 * 31 xor 5 * 39 xor 6 * 47 xor 7 * 55 xor 1 * 63
- (4,8) = 1 * 8 xor 2 * 16 xor 3 * 24 xor 4 * 32 xor 5 * 40 xor 6 * 48 xor 7 * 56 xor 1 * 64

Figure 3: Sample Results of Matrix Multiplication

With an 8×8 block matrix, we must determine a new MixColumns matrix M. M must be invertible under GF(2^8) with the same irreducible polynomial as AES, $x^8 + x^4 + x^3 + x^1 + x^0$. Determining a sufficient matrix in terms of both security implications and invertibility will likely prove the most challenging aspect of this project. Figure 1 shows two possible matrices derived from extending the 4×4 from AES in different ways. The first matrix simply extends the 4×4 without changing the starting or ending point of the matrix. The second matrix ensures that the two rows that start with 1 stay in the middle of the matrix. Both of these matrices were tested to ensure that they have diffusion. To test this, a sample test matrix was created and multiplied with the Possible M Matrices, and the equations created by the multiplication were examined to see that diffusion occurred. This multiplication and a few of the validation equations are shown in Figure 2 and Figure 3.

$$\begin{bmatrix} 1 & 1 & 4 & 1 & 8 & 5 & 2 & 9 \\ 9 & 1 & 1 & 4 & 1 & 8 & 5 & 2 \\ 2 & 9 & 1 & 1 & 4 & 1 & 8 & 5 \\ 5 & 2 & 9 & 1 & 1 & 4 & 1 & 8 \\ 8 & 5 & 2 & 9 & 1 & 1 & 4 & 1 \\ 1 & 8 & 5 & 2 & 9 & 1 & 1 & 4 \\ 4 & 1 & 8 & 5 & 2 & 9 & 1 & 1 \\ 1 & 4 & 1 & 8 & 5 & 2 & 9 & 1 \end{bmatrix}$$

Figure 4: Whirlpool Hashing Function Mix Rows Matrix

$$\begin{bmatrix} 245 & 176 & 245 & 23 & 49 & 42 & 50 & 120 \\ 120 & 245 & 176 & 245 & 23 & 49 & 42 & 50 \\ 50 & 120 & 245 & 176 & 245 & 23 & 49 & 42 \\ 42 & 50 & 120 & 245 & 176 & 245 & 23 & 49 \\ 49 & 42 & 50 & 120 & 245 & 176 & 245 & 23 \\ 23 & 49 & 42 & 50 & 120 & 245 & 176 & 245 \\ 245 & 23 & 49 & 42 & 50 & 120 & 245 & 176 \\ 176 & 245 & 23 & 49 & 42 & 50 & 120 & 245 \end{bmatrix}$$

Figure 5: Inverse of Whirlpool Hashing Function Mix Rows Matrix

Knowing that this extension of the AES matrix was not a foolproof solution, the team continued research on an 8×8 matrix with diffusion properties and found the whirlpool hashing function. This function, which has been explained in more detail in the related works section, uses the matrix in Figure 4. This matrix is a maximum distance separable matrix, which has been shown to have very strong diffusion [3][4]. To ensure that this matrix would work within our algorithm, we needed to ensure that an inverse of this matrix existed within $GF(2^8)$, which is shown in Figure 5. After

deliberation, the team has decided to use the Whirlpool hashing matrix instead of the extension of the AES matrix, as the matrix has been proven in depth to have strong diffusion properties, whereas the extension has been informally proven to have diffusion properties.

Originally, the team was hesitant to use the whirlpool hashing function, as the inverse matrix used values that were much larger than our original MixRows matrix, leading to a substantial increase in runtime. After analysis, the team recognized that it was more computationally feasible to store the inverse table and then use it as a lookup table. Although it increases the amount of memory the algorithm uses, it is a negligible amount within the scope of this project.

Number of Rounds

When developing BES-512, one of the key design decisions involved selecting the most optimal amount of rounds for the cipher. Initially we looked at the already established AES standard, where the number of rounds increases with key size as shown by figure 6: [2]:

Key Size	Number of Rounds
128	10
192	12
256	14

Figure 6: AES Round Counts

From this we observed a simple pattern: for every 64-bit increase in key size, the round count increases by 2. This led to us to extend the table to higher key sizes, resulting in a round count of 22 for a 512-bit key as seen in Figure 7: $Key\ Size\ +\ 64 \rightarrow Number\ of\ Rounds\ +\ 2$

Key Size	Number of Rounds
256	14
320	16
384	18
448	20
512	22

Figure 7: Extended AES Round Count

However, after doing further research we learned that the number of rounds was not derived purely from a mathematical relationship with key size. Instead it was carefully chosen based on its ability to resist cryptanalysis and brute-force attacks, and its performance. The AES designers considered factors such as key and block length, cryptanalytic strength, and security margins of additional rounds. In BES-512, the complexity is further increased since we are expanding both key and block size to 512-bits. Therefore, determining the correct number of rounds is not as simple as following a pattern. It required deep theoretical analysis. We had to take into account the structure of BES-512, word size, and known cryptanalytic techniques and their complexity. For current implementations we are still experimenting with round count, and measuring its performance. We started off where AES-256 left off and are slowly increasing the round count to see which number of rounds yields

the best balance of performance while still maintaining security.

The actual details of this testing will be discussed later in the paper, but the results ended up being that around 22-24 rounds provided the strongest mode collapse, which largely matches the pattern from the original AES proposal. Another factor to take into account when calculating the optimal number of rounds is the increased cryptographic strength of attacks ever since AES was published, such as the biclique attack [5]. Despite the improvements, it remains only slightly stronger than the brute-force attack on AES, proving the advanced encryption standard's continued security, which in turn proves why additional rounds aren't yet necessary to account for modern attacks.

Activities

Implementing BES-512 in Rust

We chose to implement BES-512 in Rust for the low-level performance benefits and compiler checked memory safety (as opposed to C or C++). To exercise more control over our implementation, the project was split into two components: a crate for the command line interface and a crate with compiler optimization disabled for the algorithm itself. Additionally, while the CLI depended on third party crates like Clap and Anyhow, the algorithm had no runtime dependencies—only a set of basic compile-time macros. Each part of encryption and decryption—AddRoundKey, SubBytes, ShiftRows, MixColumns, key generation, and helper functions—were implemented in const functions marked to be always inlined and then combined into exposed functions. This setup made it much easier to reason through and exactly understand our implementation without worrying that hidden behavior could compromise our efforts at security such as remaining constant time regardless of input value.

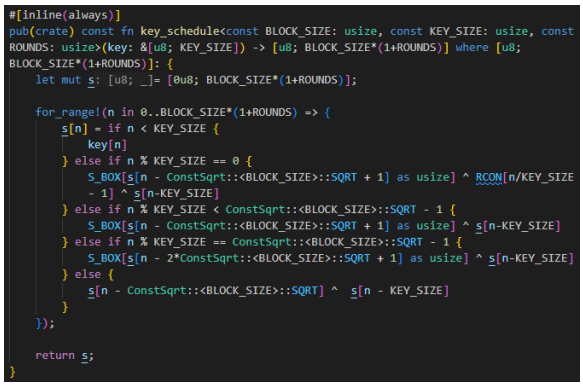
```rust
pub const fn encrypt<'a, 'b, 'c, const BLOCK_SIZE: usize, const KEY_SIZE: usize, const
ROUNDS: usize>(bytes: &'a mut [u8; BLOCK_SIZE], key: &'b [u8; KEY_SIZE], matrix: &'c
[u8; BLOCK_SIZE]) -> &'a mut [u8; BLOCK_SIZE] where [u8; BLOCK_SIZE*(1+ROUNDS)]: {
    let keys: &[u8; _] = &key_schedule::<BLOCK_SIZE,KEY_SIZE,ROUNDS>(key);

    add_round_key(bytes, key: &take_fixed(bytes: keys, offset: 0));

    for_range!(r in 1..ROUNDS => {
        sub_bytes(bytes, &S_BOX);
        shift_rows::<direction::Left,_>(bytes);
        mix_columns(bytes, matrix);
        add_round_key(bytes, &take_fixed(keys, r*BLOCK_SIZE));
    });

    sub_bytes(bytes, &S_BOX);
    shift_rows::<direction::Left,_>(bytes);
    add_round_key(bytes, key: &take_fixed(bytes: keys, offset: ROUNDS*BLOCK_SIZE));

    return bytes;
}
```

Figure 8: Abstracted Encryption Function

In order to fairly compare performance against AES, we parameterized the entire project to support arbitrary key size, block size, and round counts. These were const generic parameters which we could then easily specialize to get any of AES-128, AES-192, AES-256, or BES-512. Note that while normally discussed in terms of bits, the block and key size are represented as a number of bytes in our code (e.g. AES-128 would have a key size of 16 not 128 in the code). Additionally, using const generics for this did necessitate using experimental features on Nightly Rust and some design idiosyncrasies. No dynamic memory allocation was used, and wherever possible, static allocation was avoided. While passed in a one-dimensional array of bytes, internally, most functions logically treated the block as a column major N by N

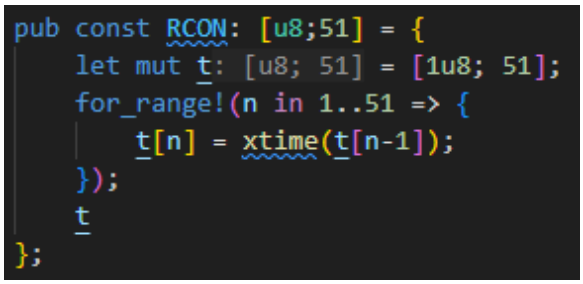
matrix where N was the square root of the block size (4x4 for AES, 8x8 for BES). The block was evolved in place via mutable reference to prevent unnecessary copying.

Key Schedule



```rust
#[inline(always)]
pub(crate) const fn key_schedule<const BLOCK_SIZE: usize, const KEY_SIZE: usize, const
ROUNDS: usize>(key: &[u8; KEY_SIZE]) -> [u8; BLOCK_SIZE*(1+ROUNDS)] where [u8;
BLOCK_SIZE*(1+ROUNDS)]: {
    let mut s: [u8; _]= [0u8; BLOCK_SIZE*(1+ROUNDS)];

    for_range!(n in 0..BLOCK_SIZE*(1+ROUNDS) => {
        s[n] = if n < KEY_SIZE {
            key[n]
        } else if n % KEY_SIZE == 0 {
            S_BOX[s[n - ConstSqrt::<BLOCK_SIZE>::SQRT + 1] as usize] ^ RCON[n/KEY_SIZE
            - 1] ^ s[n-KEY_SIZE]
        } else if n % KEY_SIZE < ConstSqrt::<BLOCK_SIZE>::SQRT - 1 {
            S_BOX[s[n - ConstSqrt::<BLOCK_SIZE>::SQRT + 1] as usize] ^ s[n-KEY_SIZE]
        } else if n % KEY_SIZE == ConstSqrt::<BLOCK_SIZE>::SQRT - 1 {
            S_BOX[s[n - 2*ConstSqrt::<BLOCK_SIZE>::SQRT + 1] as usize] ^ s[n-KEY_SIZE]
        } else {
            s[n - ConstSqrt::<BLOCK_SIZE>::SQRT] ^  s[n - KEY_SIZE]
        }
    });

    return s;
}
```

Figure 9: Key scheduling

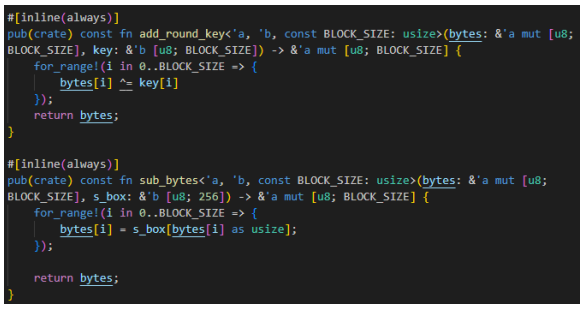


```rust
pub const RCON: [u8;51] = {
    let mut t: [u8; 51] = [1u8; 51];
    for_range!(n in 1..51 => {
        t[n] = xtime(t[n-1]);
    });
    t
};
```

Figure 10: Generation of round constants

To create a parameterized key scheduling algorithm that works for AES and BES, we abandoned the idea of "words" and did the scheduling bytewise. The first bytes were copied directly from the original key. After that, the function copies bytes and applies the round constants and round function when appropriate to match the behavior of AES's key scheduling. The ten round constants of AES are derived from repeatedly applying $x \times f(x)$ in GF(2^8) starting with 1. We generate these constants at compile-time and index an array to retrieve them. This multiplication starts to repeat itself after 52 times, so this is an upper bound on the number of rounds we support; future work should add a compile-time guard to the rounds parameter to better enforce this. In effect, with AES's key and block sizes, this code mimics the original key scheduling while being able to adapt to different configurations. If the key and block sizes are equal, then the round constant is applied each round as in AES-128 and BES-512. If the block size is smaller, then the scheduling is weaker as the round function and constants are not applied each round. This is not a flaw with our implementation; rather, it is the same issue present in AES-192 and AES-256. BES-512 avoids this issue by using a 512 bit block and key size. The weakness is only present when using our code configured to run algorithms like AES-192 or AES-256 which are known to suffer from this problem.

AddRoundKey and SubBytes



```rust
#[inline(always)]
pub(crate) const fn add_round_key<'a, 'b, const BLOCK_SIZE: usize>(bytes: &'a mut [u8;
BLOCK_SIZE], key: &'b [u8; BLOCK_SIZE]) -> &'a mut [u8; BLOCK_SIZE] {
    for_range!(i in 0..BLOCK_SIZE => {
        bytes[i] ^= key[i]
    });
    return bytes;
}

#[inline(always)]
pub(crate) const fn sub_bytes<'a, 'b, const BLOCK_SIZE: usize>(bytes: &'a mut [u8;
BLOCK_SIZE], s_box: &'b [u8; 256]) -> &'a mut [u8; BLOCK_SIZE] {
    for_range!(i in 0..BLOCK_SIZE => {
        bytes[i] = s_box[bytes[i] as usize];
    });

    return bytes;
}
```

Figure 11: AddRoundKey and SubBytes

The implementations of the AddRoundKey and SubBytes operations were trivial and not worth any further discussion beyond noting that the size of the round key must equal the block size and that the SBox—unchanged from AES—was indexed as a one-dimensional array instead of a table since the computations to mask

bits for a contiguous two-dimensional array would result in the same position anyways.

ShiftRows

```rust
#[allow(private_bounds)]
#[inline(always)]
pub(crate) const fn shift_rows<DIRECTION: direction::DirectionTy, const BLOCK_SIZE:
usize>(bytes: &mut [u8; BLOCK_SIZE]) -> &mut [u8; BLOCK_SIZE] {
    for_range!(r in 1..ConstSqrt::<BLOCK_SIZE>::SQRT => {
        let psi = gcd(ConstSqrt::<BLOCK_SIZE>::SQRT,r);
        for_range!(d in 0..psi => {
            let mut t: u8 = bytes[(r + d*ConstSqrt::<BLOCK_SIZE>::SQRT)%BLOCK_SIZE]; /
            * (r, d) */
            for_range!(n in 0..ConstSqrt::<BLOCK_SIZE>::SQRT/psi => {
                std::mem::swap(&mut t, &mut bytes[(r +
                ((direction::shift_offset::<DIRECTION>(ConstSqrt::<BLOCK_SIZE>::SQRT,
                (n + 1)*r%ConstSqrt::<BLOCK_SIZE>::SQRT) + d)
                %ConstSqrt::<BLOCK_SIZE>::SQRT)*ConstSqrt::<BLOCK_SIZE>::SQRT)
                %BLOCK_SIZE]); /* (r, (ConstMath::<N>::SQRT - (n + 1)*r + d)
                %ConstMath::<N>::SQRT) */
            });
        });
    });

    return bytes;
}
```

Figure 12: ShiftRows

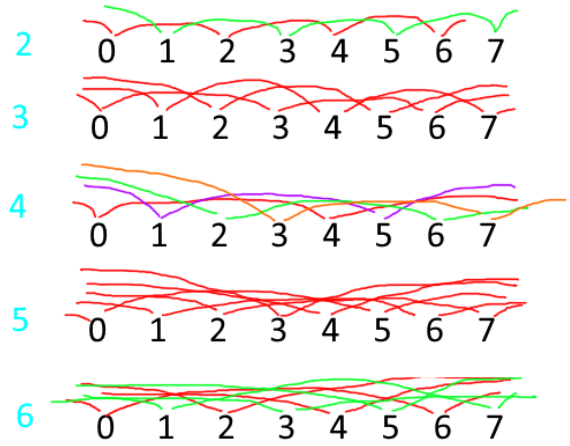


Figure 12A: The swaps required for ShiftRows starting at index 0

While conceptually simple, implementing ShiftRows in a const function without allocating extra space beyond a single temporary byte or depending on non const standard library functions proved a challenge. For a single position shift, it is sufficient to simply swap elements while walking down the line and looping back to the start; however, doing larger shifts without simply repeating single shifts is more complex. Consider a 4x4 matrix and shifting left two, it wraps around before touching every element, so the swaps must be repeated starting offset one. The number of these walks needed is equal to the greatest common divisor of the shift size and the length of data. As shown in Figure 12A for 8 bytes, 2 and 6 requires 2 walks, 4 requires 4 walks, and all odd numbers require just 1. Determining what positions to swap with was a matter of modular arithmetic. A generic parameter allowed the function to be used for left shifts in encryption or right shifts in decryption. While complicated, this allowed us to efficiently perform SwapRows without undue repetition or allocation.

MixColumns

```rust
#[inline(always)]
pub(crate) const fn mix_columns<'a, 'b, const BLOCK_SIZE: usize>(bytes: &'a mut [u8;
BLOCK_SIZE], matrix: &'b [u8; BLOCK_SIZE]) -> &'a mut [u8; BLOCK_SIZE] {
    let imm: [u8; BLOCK_SIZE] = *bytes;
    for_range!(r in 0..ConstSqrt::<BLOCK_SIZE>::SQRT => {
        for_range!(c in 0..ConstSqrt::<BLOCK_SIZE>::SQRT => {
            bytes[r + c*ConstSqrt::<BLOCK_SIZE>::SQRT] = 0;
            for_range!(n in 0..ConstSqrt::<BLOCK_SIZE>::SQRT => {
                bytes[r + c*ConstSqrt::<BLOCK_SIZE>::SQRT] ^= GF_MUL[matrix[r +
                n*ConstSqrt::<BLOCK_SIZE>::SQRT] as usize][imm[n +
                c*ConstSqrt::<BLOCK_SIZE>::SQRT] as usize];
            });
        });
    });

    return bytes;
}
```

Figure 13: MixColumns

```rust
pub const GF_MUL: [[u8; 256]; 256] = {
    const N: usize = 256;
    let mut t: [[u8; N]; N] = [[0u8; N]; N];

    for_range!(a in 1..N => {
        for_range!(b in 1..N => {
            t[a][b] = match a {
                // Identity
                1 => b as u8,
                // Mirror over diagonal
                _ if a > b => t[b][a],
                // Powers of two (1, x, x^2, ...)
                _ if (a & (a - 1)) == 0 => xtime(t[a>>1][b]),
                // All others computed via sum of x^i * b if x^i in a
                _ => {
                    let mut r: u8 = 0u8;
                    for_range!(i in 0u32..8u32 => {
                        r ^= if a & (1<<i) != 0 {
                            t[2usize.pow(i)][b]
                        } else {
                            0
                        }
                    });
                    r
                }
            }
        });
    });
    t
};
```

Figure 14: Galois field multiplication

MixColumns performs matrix multiplication in $GF(2^8)$. To support both encryption and decryption of both AES and BES, the matrix to multiply by is passed as a parameter. Instead of doing expensive computations to invert the encryption matrices, they were computed externally with Python and manually included as constants. By nature of matrix multiplication, MixColumns does necessitate taking a temporary copy of the block; however, this is done with static allocation since the block size is known at compile time. For the galois field computation, summing entries is simply xor and element wise multiplication is done using a 256 by 256 lookup table generated at compile-time. In addition to improving performance, using a LUT for the multiplication ensures that MixColumns is constant time regardless of the values in the data it handles thus eliminating a common attack against AES-like implementations. While we use the full 64 KiB (65.536 KB) table in our code to easily support any choice of matrices elsewhere, for a specific choice, the necessary portion could be cut down to only around 2-3 KiB.

Training Machine Learning Model

As mentioned in class, there are two major statistical relationships that need to be tested: confusion and diffusion. While diffusion can more or less be measured from a variety of classical metrics such as tau correlations, entropy, and the avalanche effect, confusion is not so straightforward. How does one best determine the correlation between a password and ciphertext? To that end, the team developed a series of machine learning models that treat ciphertext and password pairs as an image classification problem.

While ciphertext stems from natural language, many of the patterns that natural language processing (NLP) typically relies on do not exist on the surface. In theory, these patterns should still exist somewhere in the ciphertext but not in any immediately useful ways. Thus, the approach is to take ciphertext as byte values, normalize them, and store them in the same pattern of a grayscale image for a shape of (length, width, 1), at least at the start. This meant that the team could take advantage of convolutional neural networks (CNNs) which are extremely effective at image classification [6]. In addition, there is a type of model known as a transformer that takes advantage of self attention to better grasp global patterns in both images and natural language processing. Therefore, the two were merged to create a hybrid convolutional transformer model that uses a traditional CNN architecture with an additional self attention layer after the initial image processing (convolutional) layers. Also, the 2D convolutional layers used naturally lend themselves to this problem due to their use of a kernel as a “sliding window”. As discussed in class, AES operates on plaintext each round as a 4x4 matrix of bytes so this was the natural choice for the sizing and stepping of the conv2D layer.

The general architecture of an individual model follows this general

description. A trivial input layer is followed by the conv2D layer with a kernel size of 4x4 and a stride size of 4x4. As mentioned above, this allowed the model to effectively look at each AES byte matrix individually. Convolutional layers can excel at finding local patterns, which is still essential, but in order to understand the impact of a key/password there must be a better way to see global relationships. After the conv2D layer there is a batch normalization layer. The main thing to note about a batch normalization layer is that during training it only normalizes data based on the current batch/input, during inference however it uses a moving average derived from all of the training data which again helps capture global patterns. An activation layer using the rectified linear unit activation function (RELU) follows normalization. RELU is a very standard activation function, often utilized in a variety of models and heavily used in the Colorado School of Mines machine learning course. In general, these activation functions introduce the non-linearity that models need to effectively learn complex relationships (despite having linear in the name, RELU still accomplishes this)[7]. While there is still some debate about the order of normalization and activation, the group's models were not affected very much either way. The use of RELU concludes the image processing portion. The next layer is the transformer/self-attention portion. First is a simple reshape layer that transforms our 3D tensor into a 2D tensor. This newly reshaped tensor is then put through an embedding layer which is widely used in NLP [8]. These layers serve to take complex features and boil them down into complex feature vectors, which help subsequent layers understand ordered spatial relationships between values instead of simply seeing them individually. Next is a MultiHeadAttention layer that, when implemented, is really SelfAttention. While the inner machinations of self attention are beyond the scope of this paper, here is a brief synopsis: For a simple single head attention layer each token from the embedding layer is scored based on its relationship to all other tokens in that layer. These relationship weights are then used to represent the data in new ways. These layers take three arguments for value, key, and query but for the project's model these are all set to the embedded layer, making it self attention [9]. In NLP this is how a model can decide which words add the most meaning to a sentence, which are the exact types of patterns the group intends to find, even without seeing the words themselves. Then a multihead attention layer is just extrapolating the single headed layer such that as many sets of results are produced as there are heads. After the attention, a dropout layer is added which randomly sets values to 0 based on the rate given, but only during training. This type of layer helps prevent overfitting which both CNNs and transformer models can be particularly prone to. In fact, a second dropout layer was initially present after the convolutional layer but performance was better with this approach. This is potentially because self attention and transformers typically need as much data as they can get, so it may have benefitted from not having dropout before that portion of the model. The final portion

is classification. There are two routes typically taken to convert multidimensional data to a flat vector of probabilities that are interpreted for classification: one uses averages and the other maximums. Averages can help with the overall understanding of a set of values but maximums can help pick out the most significant. On the surface, maximums sound very useful especially since the last operation was dealing with the relative importance of tokens but they can also be sensitive to outliers. The group's implementation is with GlobalAveragePooling and GlobalMaxPooling layers due to their use in typical CNN workflows [10]. Not enough testing was conducted to see if this helped or over complicated the model but the approach was to simply use both concatenated together as a single layer. Then before the final dense layer for the output, a second dropout layer was added in case the combination layer contributed to overfitting. One could spend a lifetime tweaking the layer types, ordering, and hyperparameters only to get a black box result whose methodology is still not well known, but this configuration ultimately provided a decently performing model for the purpose of this project while still retaining some semblance of logical intention: look at the properties of individual byte matrices and then determine how they interact with each other.

Any data scientist knows that the model is at most half the battle. How the data is handled will have enormous impacts on how the model behaves. We understood that ciphertext still retains the patterns of language but not in any obvious way on its own. This is what led the group towards turning this into an image classification problem. The classic example is thinking about how human brains know there are patterns in images that can be used to tell flower species apart but directly describing those patterns to a model is rather difficult. Since AES operates on bytes as a matrix and matrix representations of values, image processing practices can naturally be employed due to the similarity in structure. Another aspect to consider is the practicality and usefulness of the data. While the initial roadmap included using randomly generated files, the group wanted the comparisons to be as close to realistic as possible so the group instead generated semi-random human readable text similar to what would actually be encrypted during real world use. To generate this dataset the team utilized the Ollama platform and the Gemma:27b large language model (LLM) to essentially roleplay as a secret agent. An initialization prompt was used to generate the first letter in a conversation and then that letter would be fed back in with the additional requirement that the LLM responds to it while staying in character. While the actual writing would not win any Pulitzer's it was still unique pieces of human readable writing that was usable. Instead of one long chain of letters, this process was repeated 40 times with each exchange lasting 5 letters for a grand total of 200 files.

At this point the model, dataset, and data embedding scheme have been discussed but there is still more to address. It seemed the most straightforward to include AES-128 and AES-256 in the analysis to provide control results for the model's performance. However, in addition to

realism, how is the comparison kept fair? There are three different algorithms with two varying block sizes and three different key sizes between them. First to be addressed is the block sizes. The input size for all algorithms was set to be three blocks of 512 bits and padded using iso7816 through PyCryptodome which allowed setting the blocksize to 512. However, this meant an operating mode had to be selected. To maintain the realism of the analysis, the team used the CBC operating mode. CBC needs an initialization vector (IV), so to simplify the process at the cost of some realism a constant IV was used while encrypting all values. To actually generate the IV, a static phrase “static iv” was used and processed via SHAKE128 since it had variable output size and allowed the use of the “same” value across all algorithms. Even though each end value was different, it at least all stemmed from the same value using the same process. This technique was then used as well on the encryption keys using a set of 4 passwords: “Andromeda”, “Neptune”, “Jupyter”, and “Saturn”. Finally, every 3 block chunk in the message dataset was encrypted with every password for all 3 algorithms and then formatted and normalized into a grayscale image format. This resulted in a sizable dataset of 57,600 data points to train, validate and test on. Once trained the performance of the models was found to be poor, which is a tricky situation. On one hand this should be a very difficult problem if all of the algorithms are functioning properly, but on the other hand crude performance alone is not conclusive evidence as maybe the model was simply poor. In an attempt to combat this, a second image channel was added to the data points similarly to how RGB images are formatted if there is a forgotten color channel in an image. This channel was used to store the normalized, padded, plaintext byte values such that the models could simulate a “known plaintext attack” in theory. The next section will cover the results stemming from this change.

Results

The following is the team’s initial statements about how results were to be navigated:

> By the end of the project the team will have created a 512 bit variant of AES taking advantage of a larger key size and more rounds such that AES-512 will be more secure than the AES-128 or AES-256 versions. AES-512 will be tested primarily through direct comparisons to both 128 and 256 variants under several modes of operation such as ECB, CBC, OFB, GCM, and CTR. A major point of focus for testing is the spatial statistics of the ciphertext, plaintext, and key to ensure that AES-512 still has sufficient confusion and diffusion properties in addition to a general flattening of single, double, and triple letter frequencies. This testing set will comprise human readable texts as well as Monte Carlo (or similarly) generated texts. Simpler to test, although still essential to AES operation is ensuring that the avalanche effect is still present in AES-512. Another consideration given the larger matrices introduced in AES-512 is

accessing any vulnerability to timing attacks via code analysis. In addition to self-made tests and statistical comparisons the team is inquiring about gaining access to NIST's Automated Cryptographic Validation Testing System (ACVTS) for a rigorous industry standard testing environment. In the event that AES-512 has any shortcomings that could not be fixed before the final deadline the team will still identify where, how, and why the issue surfaced and provide our best idea for resolution.

Please note that ACVTS was deemed unfit for this project as it did not provide a way to test homebrew algorithms like the group's BES algorithm. There was only support for established algorithms and their variants. As mentioned earlier, the void for a testing suite was filled by the team's built from scratch ML-based verification suite. The following is the discussion of the team's obtained results:

First, it is useful to observe what the models can see utilizing an autoencoder set up using the same foundational architecture as the predictive models. An auto encoder is just a type of neural network used typically in dimensionality reduction where a large model takes an input down to the desired size and then attempts to reconstruct it. For training, it uses the same data as input and output. The first layers, up to and including the desired dimensions, can then be extracted and used on their own to reduce new data. For the purpose of this project, this is just used as a visualization tool. Note that because of this process the actual meaning of what each axis represents is obscured and only has meaning to the auto encoder, but since the autoencoder uses the same front half as the predictive models, it gives a general idea of the problem's difficulty.

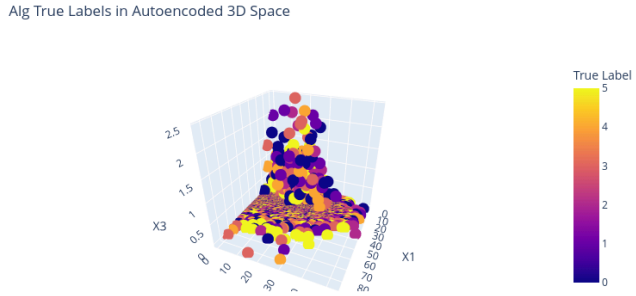


Figure 15: Autoencoded Data Set

Above is the autoencoded image of the data set colored by the algorithm used to generate that data point's ciphertext. It is quite apparent that there is no distinct grouping that can be picked out based on algorithm alone; coloring based on password is not any better. However, an initial step of the analysis is to simply attempt to train a model to classify what algorithm was used during the encryption with the results shown via confusion matrix below:

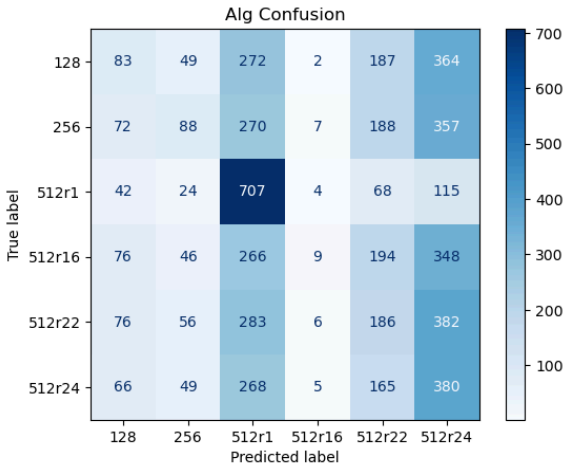
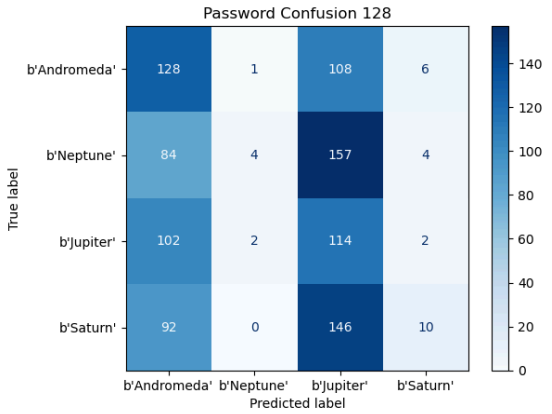
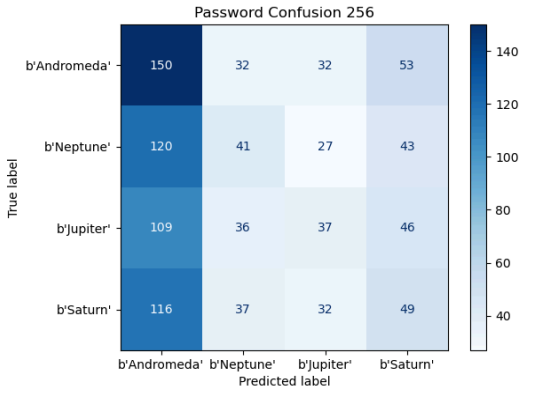


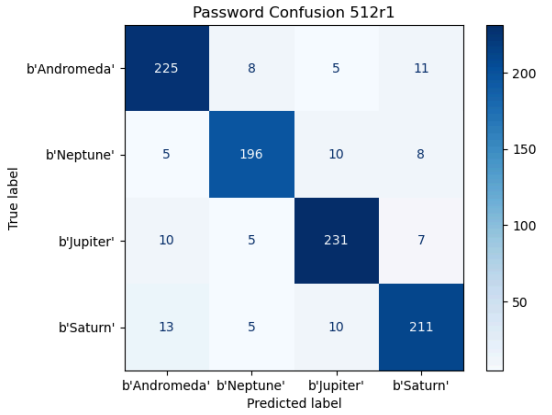
Fig 16: Confusion Matrix of Attempted Classification

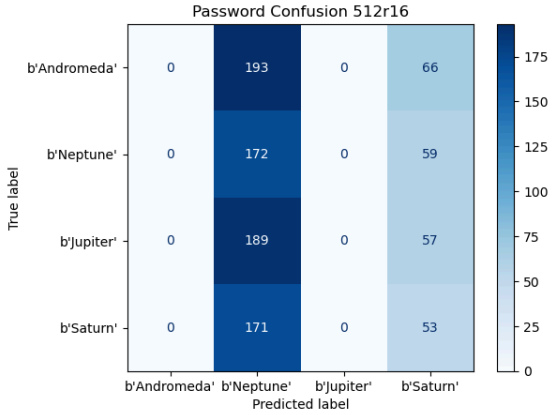
This plot shows the true value on the left axis and the bottom axis shows what the model predicted such that correct guesses appear on the diagonal. The coloration just reflects the number of guesses that landed in that square. Note that earlier only 3 algorithms were mentioned despite the presence of 6 labels. This is due to the inclusion of several different round values for BES such that 512r1 refers to a single round of BES and 512r16 refers to 16 rounds of BES etc. The general idea of this test was to see if there were any simple spatial differences that the model could detect. It can be seen that there was only ever success in picking out the single round of BES variant while the others are indistinguishable. Some values were picked more than others, but this is likely due to a slightly larger presence of that value in the training data. This is a phenomenon known as mode collapse which typically happens when the best a model can do is just predict the mode of the training data. This is not as severe a case as the earlier versions of the models whose results were seen during the in-class presentation.

The next results are the password classification confusion matrices. These models were trained on subsections of the dataset specific to each algorithm type so the only values changing would be the plaintext and the password used.









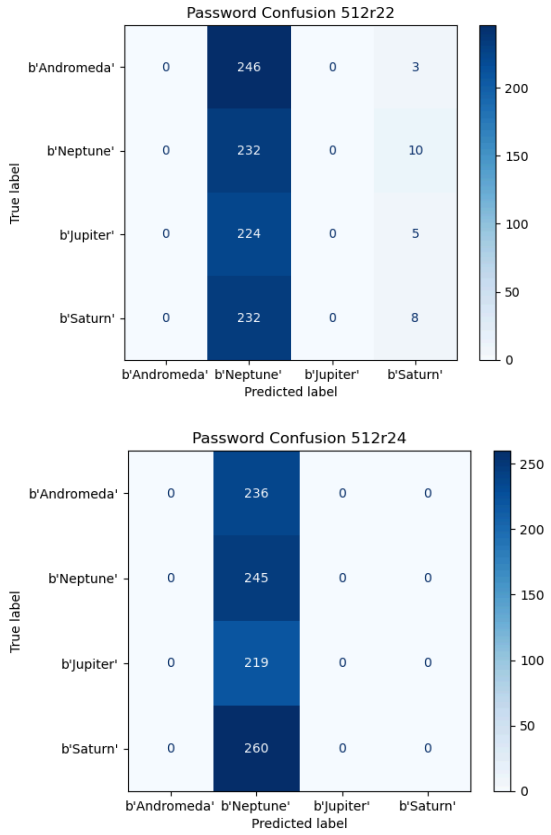


Fig 17: Confusion Matrices of Password Classification

Mode collapse emerges to some extent in all but one model. The team was aware that a single round would not be enough to ensure proper confusion so the fact that the model managed to predict the correct password label (not exact value)

90% of the time is a great indicator for the credibility of this approach. The standardized algorithms as well as BES for rounds greater than 1 all proved very resilient to the classification model. Between 22 and 24 rounds the strongest mode collapse was observed which, while not exhaustive, is a good indicator that it is a sufficient number of rounds.

The focus then shifts to the confusion properties of BES. To quantify this, the following were used: a simple byte frequency histogram, Shannon entropy, Kendall's Tau Correlation, and the avalanche effect. Each metric was calculated per each plaintext/ciphertext pair and then averaged.

First is a zoomed in portion of the byte frequency histogram. The blues and purples represent values from the plaintext while the orange represents values from the ciphertext. We can clearly see a proper flattening of character frequency which also plays into the entropy metric.

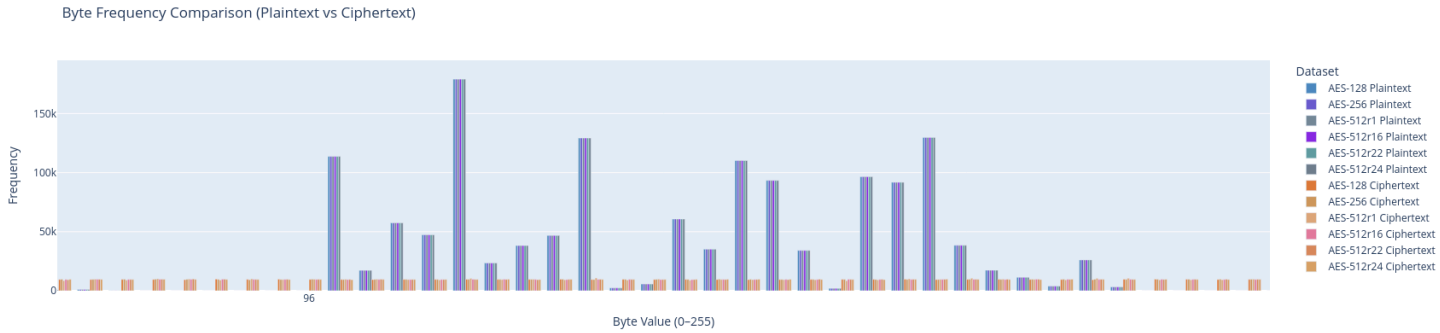


Fig. 18: Byte Frequency Histogram

The right column in each column pair is the ciphertext entropy while the left is the plaintext. Entropy is used generally to measure the randomness of a data set so the larger, the better for the ciphertext [11]. Nearly all tested algorithms are identical (differences appear around the third decimal point) even for a singular round of BES.

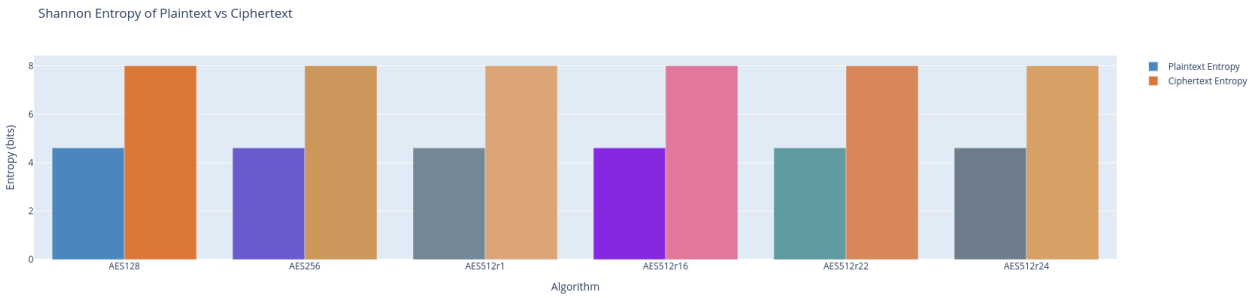


Fig. 19: Shannon Entropy of Plaintext and Ciphertext

ciphertext relative to the other byte frequencies [12]. This is useful as we know English has several characters that appear more frequently than others. Note that the y axis is extremely small so some of these differences are not as large as they may seem. That said, for rounds 16 and up of BES there is actually a smaller Tau correlation than the established variants of AES which is another indicator of a strong cryptographic algorithm.

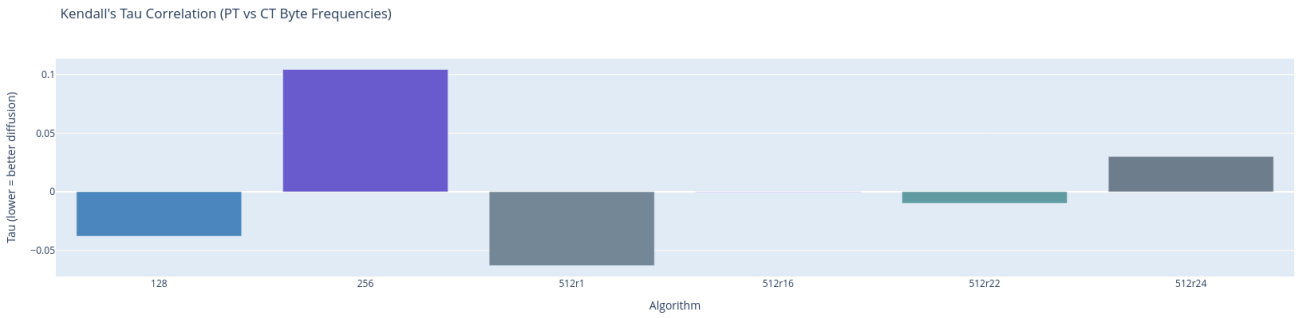


Fig. 20: Tau Correlation Plot

The plot in Fig. 20 shows the averaged Tau correlation between plaintext and ciphertext per encryption algorithm. In summary, the Tau correlation measures the statistical relationship between byte frequency rankings. That is, comparing how often one byte appeared in plaintext vs

The last metric to be discussed is the avalanche effect. Note that due to some of the considerations mentioned earlier, the displayed values are likely slightly smaller than what one would see in practice. To mitigate the effect of the constant IV, only the ciphertext differences in the second and

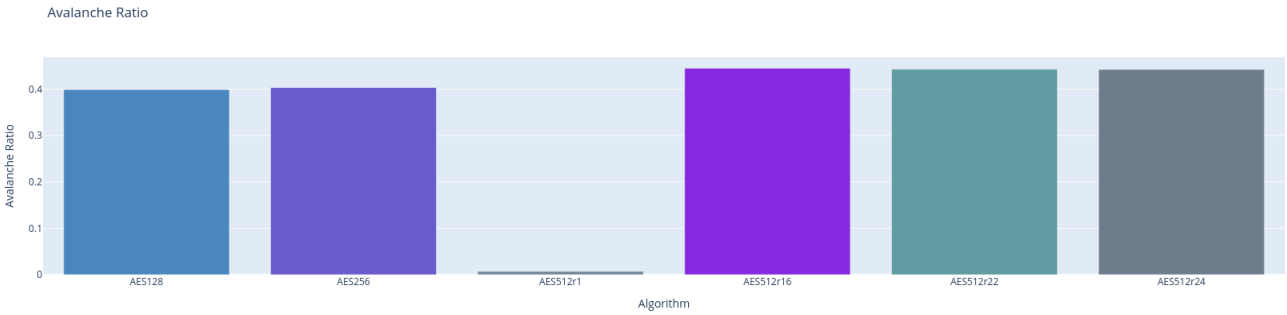


Fig. 21: Avalanche Ratio Plot

third blocks are measured. As expected, a single round performs very poorly here but once again after 16 rounds the results show a stronger avalanche in BES than in either variant of AES.

While confusion and diffusion can be difficult to measure, the team has strong evidence that BES can go toe to toe with AES while being all the more secure thanks to its increased key size.

Limitations and Future Work

There were a number of challenges and limitations that we ran into when actually implementing the 512-bit BES design, which we had to overcome, ranging from our lack of connections to the potential performance hits of doubling the key size.

One of the biggest limitations that we ran into is the lack of existing research into larger key designs for AES, as current cryptographic standards are considered good enough for the time being, staying secure against both brute force and new algorithmic attacks. Therefore, for our new design, we took inspiration from other encryption standards like Kalyna.

Another challenge faced when designing and creating BES-512 was the inevitable performance hit when it came to creating a larger key size variant of AES. To balance this out, it was important to experiment with different round numbers and determine a compromise that would sufficiently distort the plaintext, while still running in times comparable to the original design of AES.

Additional limitations included the lack of accessible, open-source testing and evaluation applications for cryptographic algorithms. While our team attempted to access some of the existing NIST validation tests, we found that they are designed for large-scale, verified cryptographic models, which is part of why we created our own suite of tests for performance evaluation.

While the current implementation of BES-512 demonstrates the feasibility of applying AES-like structures to a 512-bit block cipher, several areas in our research still remain open for further improvement and exploration.

Currently, the BES-512 implementation is written in Rust, which provides memory safe and high-level

abstractions. However, for cryptographic applications, especially in security environments, it is important to minimize timing variability. Rewriting key components and functions of BES-512 in Assembly would allow for greater control over instruction-level execution, enabling more predictable, constant-time behavior. Low-level optimization can also provide performance gains by taking advantage of hardware-specific features within instruction pipelines. Furthermore BES-512 relies on transformation matrices in its linear and non-linear layers, similar to the MixColumns step and S-Box transformations in AES. A deeper mathematical analysis of these matrices is necessary to evaluate their strengths against cryptanalytic attacks. Exploring alternative matrix configurations can provide further insight into optimizing 512-bit operations which could enhance both security and efficiency of the cipher. Lastly, the number of rounds in a block cipher directly impacts both its security and performance. While the round count in BES-512 follows the same principles as AES, the increase in block size may warrant a different optimization strategy. Future work will include evaluating how varying the number of rounds affects resistance to known attacks and execution time. This analysis could lead to a more balanced design, providing strong security without unnecessary computational overhead.

Conclusion

As quantum computing becomes more prevalent, it presents new challenges to modern day cryptography. This leads to the need for stronger and more resistant encryption algorithms. The proposed Better Encryption Standard (BES), is a variant of AES that uses a 512-bit key, aiming to solve the vulnerabilities in current AES encryption methods. By making the key size larger, other components of the algorithm are also improved such as the mixcolumns matrix, shifting row operations, and number of rounds. BES is designed to offer enhanced security against both classical and quantum attacks which is superior to the current AES algorithms. Ultimately, BES is a modern day approach to encryption with the ultimate goal of protecting information well into the quantum computing future.

Throughout the course of the project, the team has excelled at developing the theory behind the algorithm and putting it into practice through a fully functional Rust implementation. To address the team's validation needs, a novel machine learning based testing suite was constructed to verify the algorithm's performance against AES-256 and AES-128. It is exciting to note that the team's AES variant demonstrated a stronger avalanche effect and smaller Tau correlation between plaintext and ciphertext than either of the current AES implementations. These empirical findings indicate that the team has successfully created an algorithm that is more secure than the existing AES algorithms. Along with the theoretical justification of BES-512's structure, the team is confident that the algorithm is a more capable option for defending against quantum computing based attacks.

The team seeks to ensure that BES meets and exceeds current cryptographic standards using statistical analysis and validation. While the implementation of larger matrices and more rounds bring complexities and computational trade offs, the team believes the increased level of security justifies the creation of this algorithm. Future work on the project encompasses rewriting the implementation in Assembly to prevent timing attacks and conducting further analysis into the matrices and round counts that exceed the scope of the current project.

The group has performed extensive work on BES-512, from the initial research into the structure and vulnerabilities of current algorithms, to the theory behind the proposed BES algorithm, then building off of that research and developing a full fledged implementation from scratch, and finally creating a machine learning cryptographic verification suite. The team hopes that this work will inspire others to seek the constant improvement of established standards and find new approaches to security in the quantum age.

References

[1] X. Bonnetain, M. Naya-Plasencia, and A. Schrottenloher, "Quantum Security Analysis of AES," *IACR Transactions on Symmetric Cryptology*, vol. 0, no. 0, pp. 1-39, 2019.

[2] D. Evans, K. Brown, and P. Bond, "FIPS 197 Federal Information Processing Standards Publication Advanced Encryption Standard (AES)," *Advanced Encryption Standard (AES)*, Nov. 2001, doi: https://doi.org/10.6028/NIST.FIPS.197-upd1 .

[3] A. Hartikainen, T. Toivanen, and H. Kiljunen, "Whirlpool hashing function", Lappeenranta University of Technology, Lappeenranta, Finland, [Online]. Available: https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=eae4ca3441b83c14e17c6866d0214b623195bdfd

[4] P. Junod and S. Vaudenaý, "Perfect Diffusion Primitives for Block Ciphers Building Efficient MDS Matrices." Accessed: May 07, 2025. [Online]. Available: https://crypto.junod.info/sac04b.pdf

[5] "New Attack on AES - Schneier on Security," *Schneier.com*, 2011. https://www.schneier.com/blog/archives/2011/08/new_attack_on_a_1.html

[6] IBM, "Convolutional Neural Networks," *Ibm.com*, Oct. 06, 2021. https://www.ibm.com/think/topics/convolutional-neural-networks

[7] S. Tiwari, "Activation functions in Neural Networks - GeeksforGeeks," *GeeksforGeeks*, Feb. 06, 2018. https://www.geeksforgeeks.org/activation-functions-neural-networks/

[8] A. Yadav, "How Does Embedding Layer Work? - Biased-Algorithms - Medium," *Medium*, Sep. 19, 2024. https://medium.com/biased-algorithms/how-does-embedding-layer-work-eafb1c721302

[9] IBM, "What is self-attention?," *Ibm.com*, Feb. 18, 2025. https://www.ibm.com/think/topics/self-attention

[10] "Papers with Code - Global Average Pooling Explained," *paperswithcode.com*. https://paperswithcode.com/method/global-average-pooling

[11] "Shannon Entropy - an overview | ScienceDirect Topics," *www.sciencedirect.com*. https://www.sciencedirect.com/topics/engineering/shannon-entropy

[12] Datatab, "t-Test, Chi-Square, ANOVA, Regression, Correlation...," *datatab.net*, 2024. https://datatab.net/tutorial/kendalls-tau